

ReasonSTL: Bridging Natural Language and Signal Temporal Logic via Tool-Augmented Process-Rewarded Learning

Bowen Ye^{1,2} Zhijian Li² Junyue Huang¹ Junkai Ma² Xiang Yin^{1,*}

¹Shanghai Jiao Tong University

²Alibaba Group, Hangzhou, China

*Corresponding author.

Abstract

Signal Temporal Logic (STL) is an expressive formal language for specifying spatio-temporal requirements over real-valued, real-time signals. It has been widely used for the verification and synthesis of autonomous systems and cyber-physical systems. In practice, however, users often express their requirements in natural language rather than in structured STL formulas, making natural-language-to-STL translation a critical yet challenging task. Manual specification requires temporal-logic expertise and cannot scale, while prompting commercial LLM APIs incurs substantial token costs and may expose sensitive system requirements to third-party services, raising privacy concerns for industrial deployment. To address these challenges, we present REASONSTL, a tool-augmented framework that adapts local open-source language models for natural-language-to-STL generation. REASONSTL decomposes the translation process into explicit reasoning, deterministic tool calls, and structured formula construction. We further introduce process-rewarded training to supervise both tool-use trajectories and final formulas, together with STL-BENCH, a bilingual, computation-aware benchmark grounded in real-world signals. Experiments show that a 4B model trained with REASONSTL achieves state-of-the-art performance in both automatic metrics and human evaluations, demonstrating that REASONSTL provides a transparent, low-cost, and privacy-preserving alternative for formal specification drafting.

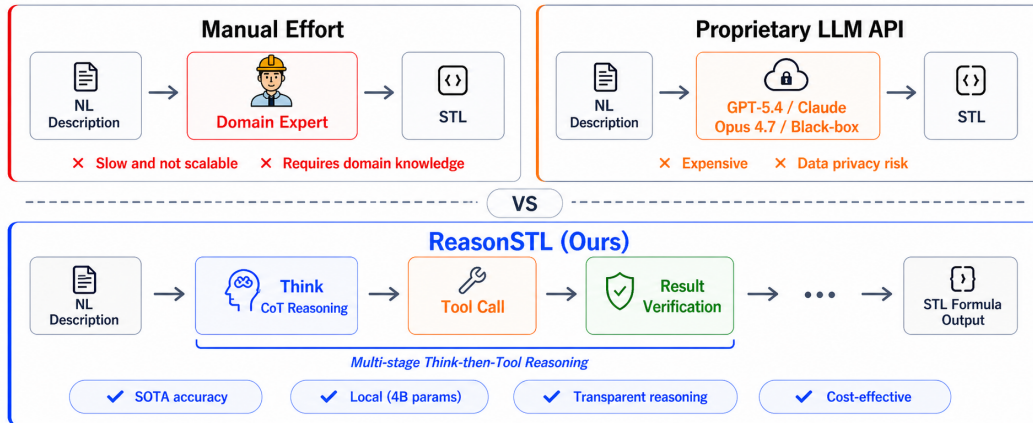


Figure 1: Comparison of three Approaches for translating NL descriptions into STL specifications. Preprint.

1 Introduction

Formal methods provide a rigorous mathematical foundation for specifying, verifying, monitoring, and synthesizing complex systems [28, 3]. By formally specifying system requirements, one can reason about safety, performance, and operational correctness with mathematical guarantees. Among formal specification languages, *Signal Temporal Logic* (STL) is particularly suitable for cyber-physical systems (CPS) and autonomous systems, as it expresses temporal properties over continuous, real-valued signals such as distance, velocity, temperature, voltage, pressure, and control inputs [17, 2]. STL has been widely applied in monitoring, verification, planning, learning, and control across domains including autonomous driving [1], unmanned aerial vehicles [25], robotics [5, 14], electronic systems [13], and collaborative robots [29]. It also serves as objectives or constraints in optimization, end-to-end learning, zero-shot task allocation, and temporal-logic-guided reinforcement learning [23, 27, 16, 24, 20].

In practice, many CPS tasks are described in natural language by engineers or domain experts, using high-level requirements that encode safety constraints, performance goals, and operational assumptions. Translating these specifications into temporal logic is therefore crucial, as it enables formal verification, automated planning, and synthesis while bridging the gap between informal descriptions and mathematically precise system behavior. Most existing NL-to-TL methods focus on Linear Temporal Logic (LTL) or Boolean specifications, which are suitable for symbolic discrete propositions but do not directly support the continuous, signal-based reasoning required for STL [4, 11, 15, 6]. This gap motivates the development of reliable NL-to-STL generation capable of handling numerical values, continuous signals, temporal intervals, and domain-specific predicates inherent in CPS requirements.

NL-to-STL translation is technically challenging. STL formulas are executable formal objects, where small errors in operator structure, predicate grounding, temporal intervals, or numerical thresholds can lead to incorrect semantics. Realistic CPS requirements often include colloquial temporal expressions, physical units, arithmetic constraints, event-triggered conditions, and domain-specific signals. For instance, phrases like “within half an hour”, “10,000 ft”, or “200 knots” must be normalized into precise temporal intervals or numerical predicates, while lexical cues such as “rise”, “drop”, or “surge” may not correspond directly to formal events. Effective NL-to-STL generation therefore requires semantic grounding, computation-aware reasoning, and structured formula construction beyond mere syntactic validity.

Translating natural-language requirements into STL formulas has traditionally relied on manual specification by engineers or domain experts. While this approach is transparent and allows precise control over the resulting formulas, it is labor-intensive and does not scale to large or complex tasks. To reduce this burden, recent works have laid important foundations for automated NL-to-STL generation. For example, DeepSTL casts the task as a neural machine translation problem and trains the network from scratch [12]; however, this approach is constrained by its from-scratch training paradigm, which restricts its generalization across diverse specifications and deployment scenarios. KGST introduces a generate-then-refine pipeline in which a fine-tuned small model first produces candidate formulae, an external knowledge base retrieves relevant examples, and a proprietary LLM API ultimately synthesizes the final STL output [8]. Nevertheless, the concurrent involvement of fine-tuned small models and proprietary LLM APIs inevitably entails substantial computational overhead and raises non-trivial privacy concerns, as sensitive data must be transmitted to external third-party services. RESTL further incorporates reinforcement learning with multi-aspect rewards, curriculum learning, and PPO-based optimization [9]. Nevertheless, despite operating entirely on local models, it directly generates STL formulae without any intermediate reasoning process, which may compromise the interpretability and correctness of the outputs. As illustrated in Figure 1, these limitations collectively underscore the need for a principled framework that is fully local, annotation-free, scalable, and capable of producing verifiable intermediate reasoning traces throughout the STL synthesis process.

To address these challenges, we present REASONSTL, a local tool-augmented framework for computation-aware NL-to-STL generation. Instead of single-step sequence prediction, REASONSTL decomposes specification drafting into explicit reasoning, deterministic computation, and structured STL construction. The model invokes tools for temporal normalization, unit conversion, arithmetic evaluation, and time-difference computation, then assembles a structured STL JSON tree from verified intermediate results. This decouples linguistic and structural reasoning from exact numerical compu-

tation, making the generation process inspectable and amenable to validation. Outcome-bounded process supervision provides feedback on both intermediate tool use and final STL correctness, encouraging the model to learn when deterministic computation is needed and how to integrate results.

We also introduce STL-BENCH, a bilingual benchmark for structured and computation-aware NL-to-STL evaluation. It covers domain-grounded CPS signals, physical units, arithmetic constraints, nested temporal structures, structured STL trees, and tool-use annotations, constructed through expert-guided templates, model-assisted requirement realization, rule-based validation, embedding-based semantic pruning, and human auditing. Experiments demonstrate that REASONSTL achieves state-of-the-art performance on both automatic metrics and human verification, providing a local, verifiable alternative to black-box API-based STL generation.

The main contributions are summarized as follows:

- **Local tool-augmented NL-to-STL generation.** We propose REASONSTL, a local open-source framework that converts natural-language requirements into STL specifications through multi-step reasoning, deterministic tool invocation, and structured formula construction. It explicitly handles temporal normalization, unit conversion, arithmetic evaluation, predicate grounding, and event-operator selection.
- **Outcome-bounded process supervision.** We introduce a process-aware training strategy that supervises both intermediate tool-use trajectories and final STL construction. By bounding the maximum intermediate reward with final-formula correctness, it reduces the risk of reinforcing plausible but semantically invalid reasoning traces.
- **Computation-aware benchmark and evaluation.** We construct STL-BENCH, a bilingual benchmark with domain-grounded CPS signals, physical units, arithmetic constraints, nested temporal structures, and tool-use annotations. Experiments on STL-BENCH and existing NL-to-STL datasets show that REASONSTL achieves state-of-the-art performance, providing a locally deployable alternative to black-box API-based specification generation.

2 Preliminaries

2.1 Signal Temporal Logic

Let $\mathcal{S} \subseteq \mathbb{R}^d$ denote the signal space, where each dimension corresponds to a real-valued CPS signal variable, and let $\mathbf{s}_{0:T} = (s_0, \dots, s_T) \in \mathcal{S}^{T+1}$ denote a finite discrete-time trace. Signal Temporal Logic (STL) provides a formal language for specifying temporal properties over such traces. We consider the following grammar:

$$\phi ::= \top \mid \pi^\mu \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \mathbf{U}_{[a,b]} \phi_2, \quad (1)$$

where π^μ is an atomic predicate induced by a function $\mu : \mathcal{S} \rightarrow \mathbb{R}$ and is satisfied at time k iff $\mu(s_k) \geq 0$. The temporal interval bounds satisfy $a, b \in \mathbb{N}$ and $0 \leq a \leq b$.

The bounded until operator is interpreted as

$$(\mathbf{s}, k) \models \phi_1 \mathbf{U}_{[a,b]} \phi_2 \quad (2)$$

iff there exists a time index $k' \in [k + a, k + b]$ such that $(\mathbf{s}, k') \models \phi_2$ and $(\mathbf{s}, k'') \models \phi_1$ for all $k'' \in [k, k']$. The standard derived operators “eventually” and “always” are defined as

$$\mathbf{F}_{[a,b]} \phi := \top \mathbf{U}_{[a,b]} \phi, \quad \mathbf{G}_{[a,b]} \phi := \neg \mathbf{F}_{[a,b]} \neg \phi. \quad (3)$$

We write $\mathbf{s} \models \phi$ as shorthand for $(\mathbf{s}, 0) \models \phi$. And STL-BENCH also includes common CPS-oriented syntactic extensions, such as threshold-crossing predicates *rise* and *fall*, and the past-time operator $\mathbf{H}_{[a,b]}$ (*historically*). We treat them as derived constructs with fixed semantics, with formal definitions provided in Appendix B.

2.2 Natural-Language-to-STL Translation

Given a natural-language requirement q , the goal is to produce an STL formula ϕ that preserves the intended temporal, logical, and numerical semantics of q . Let

$$\mathcal{D} = \{(q_i, \phi_i^*)\}_{i=1}^N \quad (4)$$

be a dataset of requirements and reference formulas. A model π_θ generates an output

$$\hat{y}_i \sim \pi_\theta(\cdot \mid q_i), \quad (5)$$

which is parsed into a predicted formula $\hat{\phi}_i$ when it satisfies the target syntax.

NL-to-STL translation differs from ordinary text generation because small structural or numerical errors can change the meaning of the specification. A valid prediction must identify signal variables, comparison operators, thresholds, Boolean structure, temporal operators, and time intervals. Many requirements also involve implicit computations, such as duration normalization, unit conversion, or arithmetic threshold calculation, before the final formula can be constructed.

Structured representation. For training and evaluation, we serialize STL formulas as JSON trees. Internal nodes represent temporal or Boolean operators, and leaves represent atomic predicates. For example,

$$\mathbf{F}_{[0,1800]}((\text{altitude} > 3048.0) \wedge (\text{speed} \geq 102.89))$$

is represented as a tree rooted at **Finally** with interval $[0, 1800]$, whose child is an **and** node over two predicates.

This representation removes ambiguity from parentheses and operator precedence, supports schema validation, and enables recursive comparison of formula structure. Importantly, the JSON representation is an evaluation interface rather than an additional semantic language: each valid JSON tree corresponds to an STL formula. A concrete schema example is given in Appendix C.

Evaluation metrics. We report two automatic metrics.

Format Accuracy measures whether the generated STL formula matches the reference at the structural level after abstracting away fine-grained predicate and numerical details. Specifically, we first parse the model output into a structured STL tree and then apply a masking function $\mathcal{M}(\cdot)$ that replaces predicate contents, temporal bounds, and numerical thresholds with abstract placeholders. Format Accuracy is defined as

$$\text{FormatAcc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[\hat{y}_i \in \mathcal{Y}_{\text{valid}} \wedge \text{Match}_{\text{STL}}(\mathcal{M}(\hat{\phi}_i), \mathcal{M}(\phi_i^*)) \right], \quad (6)$$

where $\mathcal{Y}_{\text{valid}}$ denotes the set of outputs that can be parsed into the required structured STL representation. Thus, Format Accuracy evaluates whether the model produces the correct formula skeleton, including the temporal and logical operator structure, while ignoring exact predicate grounding and numerical values.

Formula Accuracy is a stricter metric that requires the generated formula to fully match the reference formula:

$$\text{FormulaAcc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[\hat{y}_i \in \mathcal{Y}_{\text{valid}} \wedge \text{Match}_{\text{STL}}(\hat{\phi}_i, \phi_i^*) \right]. \quad (7)$$

Here, the matcher recursively compares STL formula trees, treats commutative Boolean operators as order-invariant, and applies a fixed numerical tolerance to temporal bounds and numerical thresholds. Each sample receives a binary score: a correct match is assigned 1, and an incorrect match is assigned 0. Since automatic tree matching may not capture all semantic equivalences between STL formulas, we complement it with human verification in Section 5.

3 STL-BENCH: A Computation-Aware NL-to-STL Benchmark

Benchmark overview. Existing NL-to-STL datasets have enabled early progress in data-driven specification generation, with DeepSTL providing a grammar-guided English–STL corpus [12] and STL-DivEn improving linguistic diversity through LLM-assisted augmentation and validation [8]. However, they provide limited coverage of computation-sensitive CPS requirements involving domain-grounded predicates, temporal normalization, physical units, arithmetic thresholds, event-triggered semantics, and multilingual descriptions. To address this gap, STL-BENCH is designed around three principles: *domain grounding*, where predicates are defined over physically meaningful CPS signals rather than abstract symbols; *computation awareness*, where time expressions, units, and arithmetic

Table 1: Summary of STL-BENCH construction and evaluation splits.

Property	Value	Description
Total samples	28,880	Bilingual NL-STL pairs
Languages	2	English and Chinese
Domains	6	Industrial, automotive, aerospace, robotics, environmental, electrical
Scenarios	33	Domain-specific CPS settings
Signals	41	Domain-grounded CPS variables
Complexity	1–6	Formula and tool-use complexity
Tool-use samples	73%	Require intermediate computation
parse_duration	7,337	Temporal normalization
convert_unit	5,161	Unit conversion
eval_math_expr	4,527	Arithmetic evaluation
Deduplication	Symbolic + embedding	Structural and semantic filtering
Standard split	8:1:1	Train / validation / test
Held-out split	Scenario-level	Unseen scenarios
Manual test set	~200	Human-written, template-free
Human auditing	Stratified	Language, domain, complexity, and tool type

Table 2: Comparison with representative NL-to-STL datasets.

Dataset Property	DeepSTL [12]	STL-DivEn [8]	STL-BENCH (OURS)
Language	English	English	English + Chinese
Construction	Grammar-based	Seed + LLM + validation	Scenario + template + LLM + validation
Domain grounding	No	Partial	6 domains / 33 scenarios
Signal variables	Symbolic	Mostly symbolic	CPS-semantic variables
Output format	STL string	STL string	Structured JSON tree
Intermediate computation	No	No	Explicitly annotated
Tool-use trajectory	No	No	73% samples
Complexity annotation	No	No	1–6 levels
Semantic deduplication	No	Partial	Symbolic + embedding-based pruning
Split protocol	Not emphasized	Not emphasized	Standard + held-out + manual
Human validation	No	Yes	Stratified audit

thresholds are explicitly normalized before formula construction; and *structured evaluation*, where STL specifications are represented as JSON trees to support schema validation, recursive structural matching, and diagnostic analysis of intermediate tool use.

Tables 1 and 2 summarize the benchmark statistics and its comparison with representative NL-to-STL datasets. STL-BENCH contains 28,880 bilingual NL-STL pairs across six engineering domains and 33 CPS scenarios, with 73% of samples requiring at least one intermediate computation. Compared with prior datasets, STL-BENCH evaluates not only final-formula correctness, but also predicate grounding, temporal normalization, unit conversion, arithmetic reasoning, event-operator grounding, bilingual understanding, and structured formula construction.

Construction and validation. STL-BENCH is constructed through a template-anchored and verification-driven pipeline. We first define CPS domains, scenario categories, semantic signal variables, and parameterized STL templates. Each template specifies admissible operators, signal variables, time intervals, thresholds, and optional unit systems. A strong language model is then used for controlled requirement realization: it generates engineering-style natural-language descriptions conditioned on the fixed scenario, signal semantics, numerical parameters, and target formula structure, while the reference STL formula is determined by the template and parameter assignment before text generation. This design separates formal-label construction from linguistic realization and reduces semantic drift.

During annotation, we distinguish threshold-state requirements from edge-triggered event requirements. Expressions such as “rises above”, “drops below”, or “surges past” are not automatically mapped to *rise* or *fall*; they are annotated as edge events only when the requirement explicitly specifies a transition from violation to satisfaction, or vice versa. For computation-sensitive samples, we additionally annotate tool-use trajectories, including the tool type, input arguments, expected outputs, and the final STL fields where computed values are used.

All samples are processed by rule-based validators before inclusion. The validators check JSON syntax, STL-tree well-formedness, operator arity, temporal-interval validity, predicate format, signal-variable validity, unit-conversion consistency, and alignment between tool outputs and final formula values. Invalid samples are removed or repaired only when the correction is deterministic. We

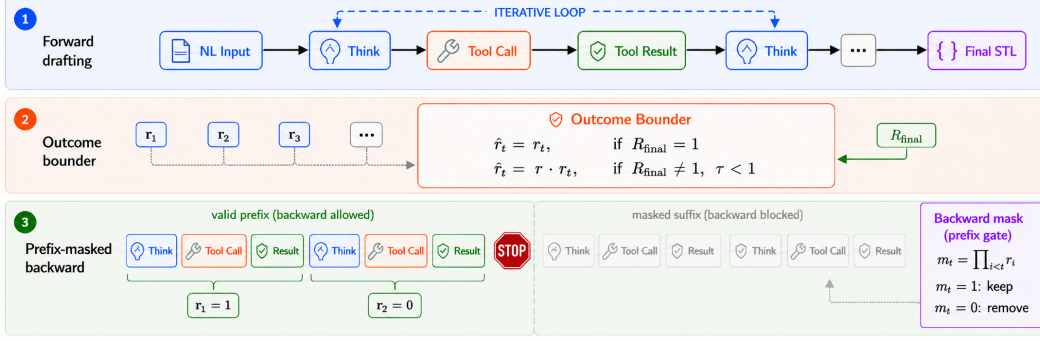


Figure 2: Overview of REASONSTL. The framework converts natural-language requirements into structured STL specifications through interleaved reasoning and deterministic tool execution, and is optimized with outcome-bounded process rewards to supervise both intermediate tool use and final formula construction.

further apply symbolic duplicate filtering over STL structures and predicate assignments, followed by embedding-based semantic near-duplicate pruning using Qwen3-Embedding-4B. Finally, a stratified subset is manually audited across languages, domains, complexity levels, and tool-use types.

Split protocol. After deduplication, we construct train, validation, and test sets using fixed split. In addition to this standard partition, we build a scenario-aware split in which a subset of scenarios from each high-level engineering domain is held out from training and used only for validation or testing. This protocol reduces direct reuse of identical domain–template combinations while preserving broad coverage for controlled model comparison.

To further assess generalization beyond the template-anchored generation process, we construct an additional manually written test set of approximately 200 requirements. These samples are authored by human annotators in both English and Chinese without using the data-generation templates, paired with manually specified structured STL formulas, and checked by the same validation pipeline. We use this set as an extra held-out evaluation and report its results together with the standard and scenario-aware splits in Section 5.

4 Method

This section presents REASONSTL, a local tool-augmented framework for computation-aware NL-to-STL generation. As shown in Figure 2, REASONSTL formulates specification generation as a structured decision process: temporal bounds, physical units, arithmetic thresholds, and event predicates are explicitly resolved through intermediate reasoning and deterministic tool calls before being assembled into a structured STL specification. The framework is trained with outcome-bounded process rewards, where intermediate rewards are capped by final-formula correctness and assigned only to valid reasoning prefixes. This encourages the model to construct correct STL formulas through faithful step-by-step computation rather than rewarding invalid intermediate trajectories.

4.1 Tool-Augmented STL Generation

Given a natural-language requirement q , the model generates a structured STL formula $\hat{\phi}$ in the JSON-tree representation defined in Section 2.2. A rollout is written as

$$y = (z_1, u_1, o_1, \dots, z_M), \quad (8)$$

where z_m is the model-generated segment at stage m , u_m is an optional tool call, and o_m is the deterministic tool output. The final segment contains the predicted STL JSON. REASONSTL uses a compact typed tool set

$$\mathcal{T} = \{\text{parse_duration}, \text{convert_unit}, \text{eval_math_expr}, \text{calc_time_diff}\}, \quad (9)$$

covering temporal normalization, unit conversion, arithmetic evaluation, and time-difference computation. This interface assigns exact numerical computation to deterministic tools, while leaving semantic interpretation, tool selection, and STL composition to the language model.

4.2 Process-Rewarded Optimization

Tool-augmented generation makes each rollout decomposable into reasoning segments, tool calls, tool outputs, and final STL construction. This enables fine-grained supervision, but also creates a structured credit-assignment problem: an incorrect formula may arise from invalid tool selection, malformed arguments, inconsistent tool outputs, unjustified event-operator grounding, ill-formed JSON, or an incorrect STL tree. REASONSTL therefore combines outcome-level STL rewards with outcome-bounded process rewards and prefix-masked backward supervision.

For a requirement q , we sample a group of G rollouts from the old policy $\pi_{\theta_{\text{old}}}$. Each rollout y_i is parsed into K_i intermediate stages and a final structured STL prediction $\hat{\phi}_i$. A deterministic validator assigns a binary local correctness score

$$c_{i,k} \in \{0, 1\}, \quad k = 1, \dots, K_i, \quad (10)$$

to each intermediate stage, checking tool validity, argument format, executability, output consistency, and semantic correctness. Event operators such as *rise* and *fall* are rewarded only when the requirement explicitly specifies a transition, rather than being inferred from surface lexical cues.

The final STL prediction is evaluated by recursive tree matching. Let

$$S_i^{\text{tree}} = \text{TreeMatch}(\hat{\phi}_i, \phi^*) \in [0, 1], \quad (11)$$

where the matcher `TreeMatch` compares operators, intervals, predicates, and subformulas, with order-invariant matching for commutative Boolean operators and numerical tolerance for time bounds and thresholds. If the output cannot be parsed into a valid JSON tree, we set $R_i^{\text{fnt}} = 0$ and $S_i^{\text{tree}} = 0$. To preserve a strict preference for exact correctness, non-perfect tree matches are capped:

$$R_i^{\text{cnt}} = \begin{cases} 1, & S_i^{\text{tree}} = 1, \\ \kappa S_i^{\text{tree}}, & S_i^{\text{tree}} < 1, \end{cases} \quad \kappa < 1. \quad (12)$$

The outcome reward and final exact-correctness indicator are defined as

$$R_i^{\text{out}} = R_i^{\text{fnt}} R_i^{\text{cnt}}, \quad C_i^{\text{final}} = \mathbb{I}[R_i^{\text{fnt}} = 1 \wedge S_i^{\text{tree}} = 1], \quad (13)$$

where $R_i^{\text{fnt}} \in \{0, 1\}$ indicates JSON/schema validity.

We next bound process rewards by final-formula correctness. Even if an intermediate step is locally plausible, it should not receive full credit when the final STL formula is incorrect. Meanwhile, because later stages depend on earlier computations, process supervision is applied only to the valid prefix of a rollout. The effective process reward is

$$R_{i,k}^{\text{proc}} = \underbrace{\prod_{\ell < k} c_{i,\ell}}_{\text{prefix mask}} \cdot \underbrace{(C_i^{\text{final}} + \tau(1 - C_i^{\text{final}}))}_{\text{outcome bounder}} \cdot c_{i,k}, \quad 0 < \tau < 1, \quad (14)$$

where the empty product is defined as 1. Thus, exact final correctness allows valid intermediate stages to receive full process reward, while incorrect final formulas cap the maximum attainable process reward. Once an intermediate stage fails, all subsequent dependent stages are masked from process-level backward updates.

Finally, we compute group-relative advantages separately for final outcomes and intermediate stages:

$$A_i^{\text{out}} = \frac{R_i^{\text{out}} - \text{mean}_{j=1}^G R_j^{\text{out}}}{\text{std}_{j=1}^G R_j^{\text{out}} + \epsilon}, \quad A_{i,k}^{\text{proc}} = \frac{R_{i,k}^{\text{proc}} - \text{mean}_{j \in \mathcal{G}_k} R_{j,k}^{\text{proc}}}{\text{std}_{j \in \mathcal{G}_k} R_{j,k}^{\text{proc}} + \epsilon}, \quad (15)$$

where $\mathcal{G}_k$ contains rollouts whose parsed trajectories include the corresponding aligned stage. Final STL predictions are therefore normalized against final predictions, while intermediate tool-use stages are normalized against corresponding intermediate stages. Each token inherits the advantage of its parsed segment:

$$A_{i,t} = \begin{cases} A_{i,k}^{\text{proc}}, & y_{i,t} \text{ belongs to intermediate stage } k, \\ A_i^{\text{out}}, & y_{i,t} \text{ belongs to final STL construction.} \end{cases} \quad (16)$$

We optimize the group-relative policy objective

$$\mathcal{J}_{\text{PR}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} M_{i,t} \rho_{i,t}(\theta) A_{i,t} \right], \quad \rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid q, y_{i,<t})}, \quad (17)$$

where $M_{i,t}$ masks invalid, truncated, prefix-blocked, and deterministic tool-output tokens. We use group-normalized advantages and gradient clipping for stability, without PPO-style likelihood-ratio clipping. Additional implementation details are provided in Appendix E.

5 Experiments

We evaluate REASONSTL on two benchmarks. DeepSTL [12] serves as a standard NL-to-STL benchmark for comparison with prior task-specific systems. STL-BENCH evaluates a more computation-aware setting involving bilingual requirements, structured JSON formulas, domain-grounded CPS predicates, and explicit intermediate computations.

5.1 Experimental Setup

We report *Formula Accuracy* and *Format Accuracy*, as defined in Section 2.2. Formula Accuracy measures whether the parsed and canonicalized prediction matches the reference specification, whereas Format Accuracy evaluates whether the output satisfies the required STL representation. Black-box API baselines are evaluated with carefully designed prompts, few-shot demonstrations, and explicit output-format instructions. Unless otherwise specified, API models are decoded with temperature 0.1; Kimi-K2.6 is evaluated with temperature 1.0 because lower-temperature decoding is not supported by the provider. Local models are evaluated deterministically with temperature 0.0. We provide additional standard deviations, ablations, and implementation details in Appendix F.

Local training is conducted on $8 \times \text{H20}$ GPUs, and all local evaluations are performed on a single H20 GPU. On STL-BENCH, REASONSTL first undergoes an SFT cold start to initialize the structured JSON output format and then starts reinforcement learning. REASONSTL-DIRECT is trained for 3K steps and takes approximately 20 hours, whereas the full REASONSTL model is trained for 5K RL steps and takes approximately 5 days. Detailed comparisons between SFT-only, RL-only, and SFT-initialized RL variants are provided in Appendix F.1.

5.2 Results on DeepSTL

Table 3 reports results on the DeepSTL benchmark. Since DeepSTL does not provide intermediate-computation annotations or tool-use trajectories, we evaluate REASONSTL-DIRECT, a Qwen3-4B-based direct-generation variant without explicit think traces or tool calls, to isolate the effect of task-specific adaptation and structured STL generation. REASONSTL-DIRECT achieves the best performance among all compared methods, reaching 81.43 Formula Accuracy and 0.8257 Format Accuracy. It outperforms the strongest black-box API baseline,

Claude-Opus-4.7, by more than 15 absolute points on both metrics, and improves over RESTL by 21.58 points in Formula Accuracy and 19.30 points in Format Accuracy. The large gap over the untuned Qwen3-4B backbone indicates that general instruction-following ability alone is insufficient for reliable STL generation. Overall, these results show that task-specific adaptation can move local NL-to-STL generation toward a practically usable regime, where most generated specifications are structurally valid and formula-correct under automatic evaluation.

Table 3: Performance on DeepSTL.

Method	Formula Acc.	Format Acc.
ReasonSTL-Direct	0.8143	0.8257
Claude-Opus-4.7	0.6603	0.6691
RESTL [9]	0.5985	0.6327
DeepSeek-V4	0.5685	0.5758
GPT-5.4	0.5150	0.5220
Kimi-K2.6	0.5048	0.5113
KGST [8]	0.4538	0.4939
DeepSTL [12]	0.2002	0.2916
Qwen3-4B	0.1131	0.1631

5.3 Results on STL-BENCH

Table 4 reports automatic evaluation on STL-BENCH. The base Qwen3-4B model performs poorly in both languages, showing that general instruction-following ability alone is insufficient for structured

NL-to-STL generation. Task-specific adaptation brings substantial gains: REASONSTL-DIRECT improves Formula Accuracy from 0.070 to 0.420 on English and from 0.090 to 0.330 on Chinese. The full REASONSTL model further improves to 0.510 and 0.470, respectively, indicating that explicit reasoning, deterministic tool use, and process-level supervision are beneficial for computation-sensitive requirements.

Under automatic matching, REASONSTL also outperforms carefully prompted API baselines, suggesting stronger local adaptation to schema-constrained and canonical STL generation. Since automatic matching may undercount semantically correct formulas that differ from the reference through harmless surface variations, predicate naming differences, or equivalent reformulations, we complement it with human semantic verification below.

Table 4: Automatic evaluation on STL-BENCH.

Method	Think	Tool	English		Chinese	
			Form.	Fmt.	Form.	Fmt.
Qwen3-4B	✗	✗	0.070	0.110	0.090	0.130
ReasonSTL-Direct	✗	✗	0.420	0.500	0.330	0.410
ReasonSTL	✓	✓	0.510	0.560	0.470	0.490
DeepSeek-V4	–	–	0.310	0.330	0.270	0.350
GPT-5.4	–	–	0.220	0.240	0.190	0.200
Claude-Opus-4.7	–	–	0.260	0.280	0.270	0.280
Kimi-K2.6	–	–	0.170	0.200	0.190	0.200

To assess whether strict tree matching underestimates semantically valid but non-canonical predictions, we conduct human verification on 100 randomly sampled cases from each language split. A prediction is judged correct if it preserves the intended requirement semantics, even when it differs from the reference through benign surface variations, predicate renaming, or logically equivalent reformulations. As shown in Table 5, REASONSTL remains competitive with strong proprietary models, matching the best English result and achieving the best or tied-best Chinese result.

Table 5: Human Verification.

Method	EN	CN
ReasonSTL	0.53	0.51
DeepSeek-V4	0.50	0.51
Claude-Opus-4.7	0.53	0.49

We further evaluate on a manually written, template-free test set to examine whether models generalize beyond the formula templates used during benchmark construction. As shown in Table 6, REASONSTL achieves the highest Formula Accuracy, slightly outperforming strong API baselines while achieving higher throughput than Claude-Opus-4.7. REASONSTL-DIRECT achieves the highest throughput, making it attractive when efficiency is prioritized, although its lower accuracy indicates that explicit reasoning and tool execution remain important for more reliable specification drafting.

Table 6: Results on the manually written test set and inference throughput.

Method	Form.	Throughput (qps)
ReasonSTL	0.54	2.3
ReasonSTL-Direct	0.42	6.1
DeepSeek-V4	0.51	0.6
Claude-Opus-4.7	0.53	2.2

5.4 Diagnostic Analysis

Beyond aggregate accuracy, we analyze representative failures to reveal method-specific error patterns. Carefully prompted API models often hallucinate operators, mis-handle numerical conversions, or confuse signal grounding, whereas REASONSTL mainly fails under ambiguous event semantics or non-canonical predicate grounding. Detailed analyses and SFT/RL ablations are provided in Appendix F.

6 Conclusion

This paper presented REASONSTL, a local tool-augmented framework that transforms natural-language requirements into STL specifications through explicit reasoning, deterministic tool use, and structured formula construction. We further introduced outcome-bounded process supervision to jointly guide intermediate tool-use trajectories and final STL construction, while reducing the risk of reinforcing plausible but semantically invalid reasoning traces. We also introduced STL-BENCH, a bilingual computation-aware benchmark covering domain-grounded signals, physical units, arithmetic constraints, nested temporal structures, intermediate computation, and structured STL trees. Experiments show that REASONSTL achieves state-of-the-art automatic matching performance and remains competitive with strong proprietary models under human verification. These results highlight local tool-augmented generation with process-aware supervision as a practical and privacy-conscious direction for reliable NL-to-STL specification acquisition.

References

- [1] N. Arechiga. Specifying safety of autonomous vehicles in signal temporal logic. In *2019 IEEE Intelligent Vehicles Symposium (IV)*, pages 58–63. IEEE, 2019.
- [2] E. Bartocci, J. Deshmukh, A. Donzé, G. Fainekos, O. Maler, D. Ničković, and S. Sankaranarayanan. Specification-based monitoring of cyber-physical systems: a survey on theory, tools and applications. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, pages 135–175. Springer, 2018.
- [3] C. Belta and S. Sadraddini. Formal methods for control synthesis: An optimization perspective. *Annual Review of Control, Robotics, and Autonomous Systems*, 2(1):115–140, 2019.
- [4] A. Brunello, A. Montanari, and M. Reynolds. Synthesis of ltl formulas from natural language texts: State of the art and research directions. In *26th International symposium on temporal representation and reasoning (TIME 2019)*, pages 17–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2019.
- [5] M. Charitidou and D. V. Dimarogonas. Distributed mpc with continuous-time stl constraint satisfaction guarantees. *IEEE Control Systems Letters*, 8:211–216, 2024.
- [6] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan. Nl2tl: Transforming natural languages to temporal logics using large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15880–15903, 2023.
- [7] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *International Conference on Computer Aided Verification*, pages 383–396. Springer, 2023.
- [8] Y. Fang, Z. Jin, J. An, H. Chen, X. Chen, and N. Zhan. Enhancing transformation from natural language to signal temporal logic using llms with diverse external knowledge. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10446–10458, 2025.
- [9] Y. Fang, Z. Jin, J. An, H. Chen, X. Chen, and N. Zhan. Restl: Reinforcement learning guided by multi-aspect rewards for signal temporal logic transformation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30682–30689, 2026.
- [10] F. Fuggitti and T. Chakraborti. Nl2ltl—a python package for converting natural language (nl) instructions to linear temporal logic (ltl) formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 16428–16430, 2023.
- [11] N. Gopalan, D. Arumugam, L. L. Wong, and S. Tellex. Sequence-to-sequence language grounding of non-markovian task specifications. In *Robotics: Science and Systems*, volume 2018, 2018.
- [12] J. He, E. Bartocci, D. Ničković, H. Isakovic, and R. Grosu. Deepstl: from english requirements to signal temporal logic. In *Proceedings of the 44th International Conference on Software Engineering*, pages 610–622, 2022.
- [13] Z. Kong, A. Jones, and C. Belta. Temporal logics for learning and detection of anomalous behavior. *IEEE Transactions on Automatic Control*, 62(3):1210–1222, 2016.
- [14] L. Lindemann and D. V. Dimarogonas. Control barrier functions for signal temporal logic tasks. *IEEE control systems letters*, 3(1):96–101, 2018.
- [15] J. X. Liu, Z. Yang, I. Idrees, S. Liang, B. Schornstein, S. Tellex, and A. Shah. Lang2ltl: Translating natural language commands to temporal robot task specification. *arXiv preprint arXiv:2302.11649*, 2023.
- [16] R. Liu, A. Hou, X. Yu, and X. Yin. Zero-shot trajectory planning for signal temporal logic tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [17] O. Maler and D. Nickovic. Monitoring temporal properties of continuous signals. In *International symposium on formal techniques in real-time and fault-tolerant systems*, pages 152–166. Springer, 2004.

- [18] Y. Mao, T. Zhang, X. Cao, Z. Chen, X. Liang, B. Xu, and H. Fang. NI2stl: Transformation from logic natural language to signal temporal logics using llama2. In *2024 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE International Conference on Robotics, Automation and Mechatronics (RAM)*, pages 469–474. IEEE, 2024.
- [19] D. Mendoza, C. Hahn, and C. Trippel. Translating natural language to temporal logics with large language models and model checkers. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–11. IEEE, 2024.
- [20] Y. Meng, F. Chen, and C. Fan. Tgpo: Temporal grounded policy optimization for signal temporal logic tasks. *arXiv preprint arXiv:2510.00225*, 2025.
- [21] S. Mohammadinejad, S. Paul, Y. Xia, V. Kudalkar, J. Thomason, and J. V. Deshmukh. Systematic translation from natural language robot task descriptions to stl. In *International Conference on Bridging the Gap between AI and Reality*, pages 259–276. Springer, 2024.
- [22] J. Pan, G. Chou, and D. Berenson. Data-efficient learning of natural language to linear temporal logic translators for robot task specification. *arXiv preprint arXiv:2303.08006*, 2023.
- [23] V. Raman, A. Donzé, M. Maasoumy, R. M. Murray, A. Sangiovanni-Vincentelli, and S. A. Seshia. Model predictive control with signal temporal logic specifications. In *53rd IEEE Conference on Decision and Control*, pages 81–87. IEEE, 2014.
- [24] N. Saxena, S. Gorantla, and P. Jagtap. Funnel-based reward shaping for signal temporal logic tasks in reinforcement learning. *IEEE Robotics and Automation Letters*, 9(2):1373–1379, 2023.
- [25] G. Silano, T. Baca, R. Penicka, D. Liuzza, and M. Saska. Power line inspection tasks with multi-aerial robot systems via signal temporal logic specifications. *IEEE Robotics and Automation Letters*, 6(2):4169–4176, 2021.
- [26] Y. Yang, S. Xiong, A. Payani, E. Shareghi, and F. Fekri. Harnessing the power of large language models for natural language to first-order logic translation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6942–6959, 2024.
- [27] B. Ye, J. Huang, Y. Liu, X. Qiao, and X. Yin. Bridging perception and planning: Towards end-to-end planning for signal temporal logic tasks. *arXiv preprint arXiv:2509.12813*, 2025.
- [28] X. Yin, B. Gao, and X. Yu. Formal synthesis of controllers for safety-critical autonomous systems: Developments and challenges. *Annual Reviews in Control*, 57:100940, 2024.
- [29] P. Yu, S. Dong, S. Sheng, L. Feng, and M. Kwiatkowska. Trust-aware motion planning for human-robot collaboration under distribution temporal logic specifications. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12949–12955. IEEE, 2024.

A Related Work

A.1 Natural Language to Temporal Logic

Natural-language-to-temporal-logic (NL-to-TL) translation has long been studied as a way to bridge informal requirements and formal reasoning. Brunello et al. [4] provide an overview of LTL formula synthesis from natural-language texts, highlighting core challenges such as linguistic ambiguity, proposition grounding, and the need to preserve formal semantics. Early neural approaches formulate the task as sequence-to-sequence translation from natural-language commands to non-Markovian task specifications, typically represented using LTL or related temporal-logic formalisms [11]. These works demonstrate the feasibility of learning mappings from language to temporal logic, but they mainly operate over symbolic propositions and discrete task structures.

More recent studies have explored data-driven and LLM-assisted NL-to-TL generation. NL2TL studies the transformation of natural-language requirements into temporal logic with the aid of large language models [6]. Pan et al. [22] investigate data-efficient learning of NL-to-LTL translators for robot task specification, while Yang et al. [26] and Mendoza et al. [19] further examine LLM-based temporal-logic translation and formal-requirement generation. These studies establish NL-to-TL translation as a broad and increasingly active research direction. However, most existing work focuses on LTL or related Boolean temporal specifications, where atomic propositions are typically symbolic and do not require explicit handling of real-valued signals, physical units, numerical thresholds, or temporal normalization.

A.2 NL-to-LTL Systems and Interactive Specification Drafting

Several systems aim to make NL-to-LTL translation more practical for users. Fuggitti et al. [10] introduce an NL2LTL system for converting natural-language instructions into LTL formulas, emphasizing tool usability and integration. Cosler et al. [7] study interactive translation from unstructured natural language to temporal-logic specifications, where user feedback is used to refine or disambiguate generated formulas. These works are closely related to human-in-the-loop formal specification drafting, as they lower the barrier between informal requirements and temporal-logic representations.

In robotics, LTL has also been widely used as a structured target language for task planning. Gopalan et al. [11] translate natural-language instructions into non-Markovian task specifications, while Lang2LTL maps robot commands to LTL formulas grounded in planning-relevant propositions [15]. Such methods show that natural language can be connected to formal task specifications for embodied agents. Nevertheless, their formal targets are primarily LTL formulas over symbolic propositions. In contrast, CPS-oriented STL generation must reason over continuous signals and numerical constraints, making the translation problem more computation-sensitive.

A.3 Natural Language to Signal Temporal Logic

Compared with NL-to-LTL, NL-to-STL generation has received relatively limited attention, despite the central role of STL in specifying real-valued and time-constrained CPS requirements. DeepSTL formulates NL-to-STL generation as a neural machine translation problem and introduces an early grammar-guided English–STL benchmark [12]. This work provides an important starting point for data-driven STL generation, but its requirements are largely grammar-controlled and offer limited coverage of realistic, computation-sensitive CPS specifications.

Beyond direct sequence-to-sequence translation, Mohammadinejad et al. propose an interactive and explainable framework for translating natural-language robot task descriptions into STL formulas [21]. Their method combines semantic parsing, pretrained language models, user-in-the-loop clarifications, and a small number of demonstrations to resolve ambiguities in natural-language commands. While this interaction-driven design improves interpretability and ambiguity resolution, it still relies on additional human feedback or demonstrations, which limits its scalability for fully automated and large-scale STL specification drafting.

KGST proposes a generate-then-refine framework in which a fine-tuned small model produces initial candidates, an external knowledge base retrieves relevant examples, and these are jointly fed as contextual prior information to a proprietary LLM API for final STL formula synthesis [8]. By leveraging knowledge retrieval and candidate pre-selection to guide the LLM API, KGST achieves

more accurate and reliable STL generation compared to purely grammar-guided approaches. RESTL further incorporates reinforcement learning into NL-to-STL generation through multi-aspect rewards, curriculum learning, and PPO-based optimization [9]. These methods improve final-formula prediction and candidate selection, but their supervision remains primarily centered on the final STL output rather than on explicit intermediate computations.

Recent work has also explored LLM-based NL-to-STL generation, including LLaMA-style models for translating natural-language requirements into STL formulas [18]. These studies suggest that large language models can serve as useful backbones for STL specification generation. However, existing NL-to-STL methods still underrepresent realistic CPS requirements involving domain-grounded signals, bilingual descriptions, physical units, arithmetic reasoning, nested temporal structures, event-triggered semantics, and verifiable intermediate computation. REASONSTL addresses these aspects through a local tool-augmented drafting workflow, outcome-bounded process supervision, and a bilingual computation-aware benchmark with structured STL trees and tool-use annotations.

B Additional STL Definitions

This section provides additional STL constructs used in STL-BENCH. Beyond standard future-time STL operators, the benchmark includes event predicates and past-time operators to represent event-triggered and history-dependent requirements that commonly arise in cyber-physical systems.

Rise and fall predicates. Given an atomic predicate π^μ , where $\pi^\mu[k]$ holds iff $\mu(s_k) \geq 0$, the derived event predicates *rise* and *fall* are defined as

$$\text{rise}(\pi^\mu, k) := \neg\pi^\mu[k-1] \wedge \pi^\mu[k] \equiv (\mu(s_{k-1}) < 0) \wedge (\mu(s_k) \geq 0), \quad (18)$$

$$\text{fall}(\pi^\mu, k) := \pi^\mu[k-1] \wedge \neg\pi^\mu[k] \equiv (\mu(s_{k-1}) \geq 0) \wedge (\mu(s_k) < 0). \quad (19)$$

Thus, $\text{rise}(\pi^\mu, k)$ captures a threshold-crossing event from false to true at time k , whereas $\text{fall}(\pi^\mu, k)$ captures the corresponding transition from true to false. These predicates are useful for requirements involving triggering conditions, such as a signal rising above an operational threshold or falling below a safety limit.

Historically and once operators. We also use the past-time operator $\mathbf{H}_{[a,b]}$, known as *historically*, to express requirements over a finite history window:

$$(\mathbf{s}, k) \models \mathbf{H}_{[a,b]}\phi \quad \text{iff} \quad \forall k' \in [k-b, k-a], (\mathbf{s}, k') \models \phi. \quad (20)$$

That is, $\mathbf{H}_{[a,b]}\phi$ holds at time k if ϕ has continuously held over the past interval $[k-b, k-a]$. We further include the derived past-time operator $\mathbf{O}_{[a,b]}$, known as *once*, defined as

$$(\mathbf{s}, k) \models \mathbf{O}_{[a,b]}\phi \quad \text{iff} \quad \exists k' \in [k-b, k-a], (\mathbf{s}, k') \models \phi. \quad (21)$$

Together, *rise*, *fall*, $\mathbf{H}$, and $\mathbf{O}$ allow STL-BENCH to include event-based and history-dependent specifications beyond standard future-time STL patterns.

C Structured JSON Schema

REASONSTL represents STL formulas as structured JSON trees rather than flat strings. The root node specifies the top-level STL operator, internal nodes encode temporal or Boolean composition, and atomic predicates appear as normalized leaf strings. This representation makes the formula hierarchy explicit and avoids ambiguities caused by parentheses, operator precedence, whitespace, and surface-level formatting. It also provides a unified interface for schema validation, recursive tree matching, subformula-level reward computation, and diagnostic analysis.

The following example illustrates the JSON-tree representation used in STL-BENCH:

```
{
  "STL": {
    "Operation": "Finally",
    "Time": [0, 1800],
    "Leftaction": null,
    "Rightaction": {
      "Operation": "and",
      "SubQueries": [
        "altitude>3048.0",
        "speed>=102.89"
      ]
    }
  }
}
```

This JSON tree corresponds to the STL formula

$$\mathbf{F}_{[0,1800]}((\text{altitude} > 3048.0) \wedge (\text{speed} \geq 102.89)). \quad (22)$$

The field `Operation` specifies the operator type, `Time` specifies the temporal interval when applicable, and `SubQueries` stores the children of Boolean operators. Unary temporal operators such as `Finally`, `Globally`, `Historically`, and `Once` use `Rightaction` to store their child formula. Binary temporal operators such as `Until` and `Since` use `Leftaction` and `Rightaction` to store the left and right subformulas. Boolean operators such as `and` and `or` use `SubQueries` to support multi-branch formulas. Atomic predicates are represented as normalized strings with canonical signal names, comparison operators, and numerical thresholds.

Table 7: Main fields in the structured STL JSON schema.

Field	Type	Description
STL	object	Root field containing the full STL tree
Operation	string	Operator name, e.g., <code>Finally</code> , <code>Globally</code> , <code>Until</code> , and
Time	list or null	Temporal interval $[a, b]$ for bounded temporal operators
Leftaction	object/string/null	Left child for binary temporal or implication-like operators
Rightaction	object/string/null	Child formula for unary operators, or right child for binary operators
SubQueries	list	Children of multi-branch Boolean operators such as <code>and</code> and <code>or</code>
Predicate leaf	string	Normalized atomic predicate, e.g., <code>altitude>3048.0</code>

Before evaluation, each prediction is parsed and canonicalized. Canonicalization includes normalizing operator names, checking required fields, converting numerical fields into a standard format, validating temporal intervals, and parsing predicate strings into signal–operator–threshold triples. If the prediction cannot be parsed into a valid JSON tree, it is assigned zero format reward and zero tree-matching score. Otherwise, the parsed tree is used for both automatic evaluation and reward computation.

The recursive tree matcher compares the predicted tree $\hat{\phi}$ with the reference tree ϕ^* from the root downward. Operator nodes must match in type, temporal intervals are compared with a fixed numerical tolerance, and predicate leaves are compared by signal name, comparison direction, and threshold value. For commutative Boolean operators such as `and` and `or`, child order is treated as

invariant: the matcher searches for the best alignment between predicted and reference children before computing the subtree score. This avoids penalizing formulas that differ only by the ordering of conjuncts or disjuncts.

Formally, the matcher returns a similarity score

$$S^{\text{tree}}(\hat{\phi}, \phi^*) \in [0, 1], \quad (23)$$

where 1 indicates an exact canonical tree match. The matcher assigns credit in a top-down manner: a node contributes to the score only if its operator type or predicate form matches the corresponding reference node, and child nodes are evaluated only under matched parent nodes. Consequently, if an internal operator is incorrect, the entire subtree rooted at that node receives no further credit, even if some descendant predicates happen to match the reference. For example, a correct threshold appearing under an incorrect temporal operator is not rewarded as an independently correct leaf. This design encourages models to recover the intended hierarchical STL structure rather than only matching isolated predicate strings. The resulting score is used in the content reward described in Section 4.2. To preserve a strict preference for exact formal correctness, non-perfect tree matches are capped by the factor $\kappa < 1$, so that partially correct trees can provide informative learning signals but cannot receive the full content reward.

This structured representation is also used to compute the automatic metrics in Section 2.2. Format Accuracy checks whether the model output can be parsed into the required schema and, when evaluating formula skeletons, whether the masked tree structure matches the reference. Formula Accuracy further requires the complete canonical tree, including predicates, temporal intervals, and numerical thresholds, to match the reference under the tree-matching protocol. Thus, the JSON schema serves not only as an output format, but also as the central representation for validation, evaluation, reward design, and error analysis.

D Additional Details of STL-BENCH

This section provides additional details on the construction of STL-BENCH, complementing the benchmark overview in Section 3. We describe the domain taxonomy, template design, model-assisted requirement realization, tool-use annotations, validation and deduplication procedures, split construction, and human auditing protocol.

D.1 Domain and Scenario Taxonomy

STL-BENCH is organized around six engineering domains: autonomous driving, robotics, industrial control, environmental monitoring, electrical systems, and aerospace systems. Each domain is associated with concrete CPS scenarios and semantically meaningful signal variables. This design encourages models to ground natural-language requirements in physically interpretable predicates rather than abstract symbols. In total, the benchmark covers 33 scenarios and 41 canonical signal variables.

Table 8: Domain and scenario taxonomy of STL-BENCH.

Domain	Scenarios	Example Signal Variables
Autonomous driving	AEB, ACC, lane keeping, parking, fuel monitoring, traction control	velocity, acceleration, steering, brake, obstacle distance
Robotics	pick-and-place, welding, collision avoidance, assembly, mobile navigation	position, distance, torque, contact force, joint velocity
Industrial control	reactor control, CNC machining, conveyor belt, hydraulic press, boiler system, structural monitoring	pressure, temperature, flow rate, load, strain, stress
Environmental monitoring	indoor climate, water quality, air quality, greenhouse, noise monitoring	humidity, CO ₂ level, pollutant concentration, pH level, noise level
Electrical systems	battery management, motor drive, power grid, solar panel, signal processing	voltage, current, frequency, power, phase, amplitude
Aerospace systems	altitude hold, takeoff landing, drone survey, satellite attitude, flight envelope, UAV delivery	altitude, airspeed, pitch, roll, yaw, heading

The canonical signal vocabulary is shared across English and Chinese requirements:

$$\mathcal{V} = \{\text{accel, acceleration, altitude, amplitude, brake, brightness, co2_level, concentration, current, density, dist, distance, flow_rate, frequency, fuel_level, heading, humidity, load, noise_level, oxygen, ph_level, phase, pitch, power, pressure, roll, rpm, speed, steering, strain, stress, temp, temperature, throttle, torque, velocity, voltage, x_pos, y_pos, yaw, z_pos, \dots}\}. \quad (24)$$

D.2 Template and Complexity Design

Samples are generated from scenario-conditioned STL templates. Each template specifies admissible signal variables, numerical ranges, temporal intervals, operator structures, optional unit systems, and optional tool-use requirements. The supported operators include Globally, Finally, Until, Since, imply, and, or, Not, Rise, Fall, Historically, and Once. Compared with a single grammar-only generator, the scenario-conditioned design provides tighter control over domain semantics, physically plausible value ranges, and computation-sensitive fields.

Table 9: Complexity levels used in STL-BENCH.

Level	Description	Sampling Ratio
1	Single operator with one atomic predicate	25%
2	Two operators with two to three predicates	25%
3	Three operators with three to five predicates	20%
4	Multi-stage temporal constraints with four to six predicates	15%
5	Deeply nested multi-branch formulas with five to seven predicates	10%
6	Highly complex formulas with six to eight predicates and chained tool reasoning	5%

The complexity level is determined by formula depth, number of temporal or Boolean operators, number of atomic predicates, and length of the tool-use chain. Levels 5–6 are designed to stress nested temporal structures, multi-signal dependencies, and chained intermediate computations.

D.3 Model-Assisted Requirement Realization

The formal label is determined before natural-language realization. For each candidate sample, the template first fixes the scenario, signal variables, operator structure, temporal intervals, thresholds, and tool-use requirements. A strong language model is then used only to realize this fixed formal specification as an engineering-style natural-language requirement. This separation reduces semantic drift because the reference STL tree is not inferred from generated text after the fact.

We use DeepSeek-Chat with decoding temperature 0.9. Each API call generates a small batch of candidate requirements from a structured prompt containing the domain description, scenario, admissible signal variables, signal ranges, target complexity level, STL JSON schema, tool-call format, and quality constraints. A candidate is retained only if it passes automatic validation, including natural-language length checks, STL-tree validity, signal-name validity, tool-call executability, and consistency between tool outputs and final STL fields.

For bilingual construction, English and Chinese requirements are generated independently under the same scenario and complexity conditions, rather than by direct translation. This preserves a shared formal schema and signal vocabulary while allowing language-specific phrasing, domain terminology, and engineering style.

D.4 Tool-Use Annotation

For computation-sensitive samples, STL-BENCH provides explicit tool-use annotations. Each annotation records the tool name, input arguments, expected output, and the target field in the final STL JSON where the computed value is used. The tool set includes:

- `parse_duration`: normalizes natural-language duration expressions into canonical time units;
- `convert_unit`: converts physical quantities into canonical units;

- `eval_math_expr`: evaluates arithmetic expressions used in thresholds or time bounds;
- `calc_time_diff`: computes time differences from temporal expressions or timestamps.

Table 10: Examples of tool-use annotations.

Tool	Input	Output	Used in STL Field
<code>parse_duration</code>	“30 minutes”	1800	Time
<code>convert_unit</code>	(10000, ft, m)	3048.0	predicate threshold
<code>convert_unit</code>	(200, kn, m/s)	102.89	predicate threshold
<code>eval_math_expr</code>	{"expression": "2*900"}	1800	time bound
<code>calc_time_diff</code>	“time interval between 2025-08-01 8:00:00 and 2025-08-01 8:15:00”	900	Time

D.5 Event Semantics

We explicitly distinguish threshold-state requirements from edge-triggered event requirements. Surface expressions such as “rises above”, “drops below”, or “surges past” are not automatically mapped to `Rise` or `Fall`. They are annotated as edge events only when the requirement specifies a transition from violation to satisfaction, or from satisfaction to violation. Otherwise, they are treated as ordinary threshold predicates. This distinction is important because an edge-triggered event and a threshold-state predicate can have different STL semantics even when their natural-language descriptions share similar lexical cues.

D.6 Validation and Deduplication

All generated samples are processed by automatic validators before inclusion. The validators check natural-language length constraints, JSON syntax, required fields, STL-tree well-formedness, operator arity, temporal-interval validity, predicate format, signal-variable validity, tool-call executability, unit-conversion correctness, arithmetic correctness, and consistency between tool outputs and final STL values.

Invalid samples are removed or repaired only when the correction is deterministic. For example, malformed atomic predicates containing conjunctions or disjunctions can be converted into Boolean subtrees, while unexecutable tool calls or inconsistent tool outputs lead to sample removal. This avoids introducing subjective corrections into the formal labels.

Deduplication is performed in two stages. First, exact and symbolic duplicates are removed using normalized natural-language strings, STL-structure hashing, and predicate assignments. Second, embedding-based semantic pruning is applied to reduce near-duplicate natural-language realizations. Requirements are encoded with Qwen3-Embedding-4B, samples with cosine similarity above 0.9 are clustered, and each cluster retains samples with higher formula complexity, clearer domain grounding, or better linguistic quality.

D.7 Data Splits and Manual Test Set

The standard train, validation, and test sets are constructed after deduplication using stratified random partitioning over language, domain, complexity level, and tool-use type. This preserves the overall benchmark distribution while reducing leakage from near-duplicate requirements. The split procedure uses a fixed random seed.

In addition to the standard split, STL-BENCH includes a scenario-aware held-out split, where selected scenarios from each high-level domain are excluded from training and used only for validation or testing. This split evaluates whether models can generalize across scenario variations rather than merely memorizing domain-template combinations.

We also construct an additional manually written test set of approximately 200 requirements. These samples are authored by human annotators in English and Chinese without using the data-generation templates, paired with manually specified structured STL formulas, and checked by the same validation pipeline. This set is used to assess generalization beyond the template-anchored generation process.

D.8 Human Auditing Protocol

A stratified subset of samples is manually audited to assess dataset quality. The audit checks natural-language/STL semantic alignment, JSON validity, tool-result correctness, bilingual terminology consistency, formula readability, and the distinction between threshold-state and edge-triggered requirements. The audited subset is sampled across languages, domains, complexity levels, and tool-use types.

STL-BENCH is not exhaustively verified sample by sample. Instead, it combines template-controlled label construction, deterministic validation, symbolic filtering, embedding-based semantic pruning, and stratified human auditing. This design balances dataset scale with formal-label reliability while leaving room for residual ambiguity or annotation noise, as discussed in Section G.

E Additional Method Details

E.1 Tool Execution Protocol

REASONSTL uses an explicit tool-execution interface to separate semantic interpretation from deterministic computation. During generation, the model may emit a `<tool_call>` block with a tool name and structured arguments. If the call is syntactically valid and belongs to the predefined tool set, the environment executes the tool and appends the returned value as a `<tool_result>` block. The model then continues generation conditioned on the original requirement, previous reasoning, and the verified tool output.

Consistent with Section 4.1, a rollout is represented as

$$y = (z_1, u_1, o_1, \dots, z_M), \quad (25)$$

where z_m is a model-generated segment, u_m is an optional tool call, and o_m is the deterministic tool output. Since tool-output tokens are produced by the execution environment rather than sampled from the model policy, they are excluded from policy-gradient updates by the token mask $M_{i,t}$.

```
<think>
The requirement uses "30 minutes", which should be converted to seconds.
</think>
<tool_call>
parse_duration("30 minutes")
</tool_call>
<tool_result>
1800
</tool_result>
```

Generation terminates when the model emits the final STL JSON answer, reaches the maximum number of tool rounds, exceeds the maximum generation length, or violates the reasoning-length constraint. Malformed tool calls, unsupported tools, invalid arguments, failed execution, or tool outputs inconsistent with the final formula mark the corresponding intermediate stage as invalid.

E.2 Stage Validation Details

Each rollout is parsed into intermediate stages and a final STL-construction stage. Intermediate stages include reasoning segments, tool calls, tool outputs, and the subsequent use of computed values. The deterministic validator assigns a binary correctness score $c_{i,k} \in \{0, 1\}$ to each intermediate stage. Table 11 summarizes the main validation checks.

For event-related predicates, surface expressions such as “rises above”, “drops below”, or “surges past” are not sufficient to trigger *rise* or *fall*. These operators are considered valid only when the requirement explicitly describes a transition from violation to satisfaction, or vice versa.

E.3 Tree Matching and Numerical Tolerance

The final STL JSON is evaluated by the recursive tree matcher described in Section 4.2. If the output cannot be parsed into a valid JSON tree, we set $R_i^{\text{fnt}} = 0$ and $S_i^{\text{tree}} = 0$. Otherwise, the matcher

Table 11: Validation checks for intermediate stages.

Stage Type	Check	Example
Tool selection	Tool name is valid and appropriate	<code>parse_duration</code> for “30 minutes”
Argument format	Arguments satisfy the tool schema	Valid unit string or arithmetic expression
Execution	Tool can be executed successfully	No parsing or conversion failure
Result consistency	Tool output matches later formula fields	30 min $\rightarrow$ 1800 sec in interval bound
Predicate grounding	Signal and threshold match requirement	Correct variable and inequality direction
Event grounding	Edge operator is semantically justified	<i>rise/fall</i> only for explicit transitions

recursively compares operators, temporal intervals, predicates, and subformulas. Commutative Boolean operators are matched in an order-invariant manner, and numerical fields use a fixed tolerance of 0.1 for temporal bounds and predicate thresholds. Non-perfect matches are capped by the content cap κ , as defined in Eq. (12).

E.4 Stage Alignment and Token Masking

For group-relative advantage estimation, final outcomes are normalized against final outcomes, while intermediate stages are normalized against corresponding intermediate stages. In practice, intermediate stages are aligned by their parsed role, such as duration parsing, unit conversion, arithmetic evaluation, event grounding, or final-value incorporation. This avoids mixing structurally different rewards when computing $A_{i,k}^{\text{proc}}$.

The token mask $M_{i,t}$ excludes four types of tokens: invalid tokens, truncated tokens, deterministic tool-output tokens, and tokens belonging to process stages blocked by the prefix mask in Eq. (14). Thus, only model-generated tokens with well-defined stage-level credit contribute to the policy update.

E.5 Optimization Procedure

Algorithm 1 summarizes the outcome-bounded process-rewarded optimization procedure. It follows the notation and reward definitions in Section 4.2.

Algorithm 1 Outcome-Bounded Process-Reward Optimization

Require: Requirement q , old policy $\pi_{\theta_{\text{old}}}$, group size G

- 1: Sample rollouts $\{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$
 - 2: **for** $i = 1, \dots, G$ **do**
 - 3: Parse y_i into intermediate stages and final STL tree $\hat{\phi}_i$
 - 4: Validate intermediate stages to obtain $\{c_{i,k}\}_{k=1}^{K_i}$
 - 5: Compute final outcome reward R_i^{out} and correctness indicator C_i^{final}
 - 6: **for** $k = 1, \dots, K_i$ **do**
 - 7: Compute process reward $R_{i,k}^{\text{proc}}$ using Eq. (14)
 - 8: **end for**
 - 9: **end for**
 - 10: Compute outcome advantages $\{A_i^{\text{out}}\}_{i=1}^G$
 - 11: Compute process advantages $\{A_{i,k}^{\text{proc}}\}$ over aligned stages
 - 12: Assign token-level advantages $A_{i,t}$ and masks $M_{i,t}$
 - 13: Update π_{θ} using the objective in Eq. (17)
-

E.6 SFT Cold Start and RL Optimization

For STL-BENCH, REASONSTL uses a short SFT cold start before reinforcement learning. The SFT stage initializes the model with the structured JSON output format and the basic think–tool interaction pattern. This improves early rollout quality by increasing the probability of parseable JSON, valid tool-call syntax, and recoverable intermediate traces. The final performance is then improved through tool-augmented reinforcement learning with outcome-bounded process rewards. Section F.1 provides controlled comparisons among SFT-only, RL-only, and SFT-initialized RL variants.

Table 12: Training hyperparameters for REASONSTL.

Hyperparameter	Value
Backbone	Qwen3-4B
Fine-tuning method	Full fine-tuning, with input embeddings frozen
Training GPUs	8×H20
Evaluation GPU	1×H20
SFT learning rate	2×10^{-5}
SFT per-GPU batch size	2
SFT gradient accumulation	4
SFT effective batch size	64
SFT optimizer	AdamW, weight decay 0.01
SFT warmup ratio	0.05
SFT max sequence length	2048
SFT epochs	1
SFT curriculum	First 30% steps use samples with tree depth ≤ 3
RL learning rate	3×10^{-6}
RL LR schedule	Warmup + cosine decay, minimum ratio 0.1
RL warmup steps	20
RL batch size	16
RL micro batch size	2
Group size G	8
Maximum tool rounds	5
Maximum generation tokens	2048
Training / test temperature	1.0 / 0.0
Top- p	0.95
RL optimizer	AdamW
Gradient clipping	1.0
RL epochs	9
Outcome-bound factor τ	0.5
Content partial cap κ	0.3
Numerical tolerance	0.1
Direct RL steps without think/tool	3K
Full RL steps	5K
Direct training time	~20 hours
Full training time	~5 days

E.7 Training Hyperparameters

The implementation uses an unclipped group-relative objective with token-level stage advantages. Stability is controlled by group-normalized advantages, outcome-bounded process rewards, prefix masking, gradient clipping, and the SFT cold start. Since the objective does not use PPO-style likelihood-ratio clipping, no clipping coefficient is introduced.

F Additional Experiments

F.1 SFT and RL Ablation

We conduct ablations to disentangle the effects of supervised cold-start training, reinforcement learning, think–tool interaction, and process-level rewards. The compared variants include pure SFT, RL from the base model, and SFT-initialized RL. For RL-based variants, we distinguish direct RL, which optimizes final-answer correctness without explicit think–tool interaction, from process-rewarded RL, which additionally supervises intermediate tool-use stages.

Table 13 shows that SFT alone improves over the base Qwen3-4B model but remains substantially below RL-based variants. This suggests that supervised imitation helps the model acquire the target output format, but is insufficient for robust computation-aware STL construction. Direct RL substantially improves both Formula Accuracy and Format Accuracy, indicating that reward-based optimization is important for adapting the model to the structured evaluation objective.

Table 13: SFT/RL ablation on STL-BENCH. Results are reported as Formula Accuracy / Format Accuracy, together with SFT and RL training steps.

Variant	Think	Tool	SFT	RL	English	Chinese
Qwen3-4B	✗	✗	0	0	0.070 / 0.110	0.090 / 0.130
SFT-only Direct	✗	✗	2K	0	0.200 / 0.240	0.240 / 0.280
RL-only Direct	✗	✗	0	5.5K	0.410 / 0.510	0.330 / 0.380
RL-only Think+Tool	✓	✓	0	8K	0.500 / 0.560	0.460 / 0.470
SFT + Direct RL	✗	✗	300	3K	0.420 / 0.500	0.330 / 0.410
ReasonSTL	✓	✓	300	5K	0.510 / 0.560	0.470 / 0.490

The comparison between RL-only and SFT-initialized RL indicates that the SFT cold start mainly improves training efficiency and stability. With only 300 SFT steps, SFT + Direct RL reaches performance comparable to RL-only Direct while using fewer RL steps. In the tool-augmented setting, RL-only Think+Tool already achieves strong performance, while SFT + Process-Rewarded RL obtains the best overall results with fewer RL steps than RL-only Think+Tool. These results support the use of SFT as a lightweight initialization strategy and indicate that the final performance gains primarily arise from reinforcement learning, explicit think–tool interaction, and process-level supervision.

F.2 Representation Study: Structured JSON vs. Flat String

We compare structured JSON representation with flat string-form STL generation under the same GPT-based API baseline. As shown in Table 14, JSON mode does not necessarily improve exact Formula Accuracy for a prompted black-box model, but it yields higher Template Accuracy, suggesting that the structured representation better preserves the high-level operator skeleton. More importantly, JSON provides a stable interface for trainable local models: the output schema is easier to validate, syntactic errors are easier to detect, and partial rewards can be assigned through recursive tree matching. Flat STL strings are more sensitive to parentheses, operator precedence, and surface formatting, making automatic evaluation and reward design less reliable. We therefore use structured JSON as the default representation for STL-BENCH and REASONSTL.

Table 14: Comparison between structured JSON and flat string STL representations under the same GPT-based API baseline.

Metric	JSON Mode	String Mode
Formula Accuracy	20.5%	22.8%
Template Accuracy	31.3%	24.0%

F.3 Error Breakdown

We categorize incorrect predictions to identify the dominant failure modes of prompted API models on STL-BENCH. The analysis focuses on whether failures arise from format control, temporal-operator grounding, predicate grounding, numerical computation, or tool-use behavior.

Table 15: Major error types of GPT-5.4 on STL-BENCH. Percentages are computed over 75 incorrect predictions after excluding reference-quality issues. Categories are not mutually exclusive.

Error Type	Count	Error Share
Temporal-operator hallucination	30	40.0%
Numerical / unit-conversion error	26	34.7%
Signal-name ambiguity	26	34.7%

The most frequent error is temporal-operator hallucination: models introduce formal event predicates such as *rise* or *fall* from surface cues such as “rises above” or “drops below”, even when the requirement describes ordinary threshold satisfaction. Numerical and unit-conversion errors motivate deterministic tool use, while signal-name ambiguity motivates signal-alias normalization and human verification.

F.4 Qualitative Error Cases

We present qualitative examples to complement the aggregate metrics and error statistics. These cases illustrate systematic failures of black-box API models, remaining limitations of REASONSTL, and cases where strict automatic matching either underestimates semantically plausible predictions or reveals residual reference-quality issues.

API failure: temporal-operator hallucination. A common failure mode of API-based models is to over-interpret surface-level motion verbs as formal STL event predicates. Consider the requirement:

If the current surges past 75 amperes, the system must historically have had stable voltage.

The reference formula treats the antecedent as a threshold-state predicate:

$$\text{imply}(\text{current} > 75, \dots). \quad (26)$$

In contrast, GPT-5.4 predicts:

$$\text{imply}(\text{Rise}(\text{current} > 75), \dots). \quad (27)$$

This prediction is semantically incorrect: “surges past” describes threshold exceedance, but does not necessarily specify a formal edge-triggered transition. In STL, *Rise* denotes a transition from predicate violation to predicate satisfaction, which is stronger than ordinary threshold satisfaction and changes the requirement semantics.

Reference-quality issue: incorrect geometric annotation. Manual inspection also reveals cases where automatic mismatches are caused by reference annotation errors rather than model errors. Consider:

Throughout the placement segment from time 20 to 80 seconds, the end-effector must stay within a 5 cm radius of the target ($X = 1800\text{mm}$, $Y = 200\text{mm}$) whenever its Z position is less than 10 cm.

The reference formula encodes the geometric condition as axis-aligned bounds:

$$\mathbf{G}_{[20,80]}(\text{imply}(\text{z_pos} < 0.1, (\text{x_pos} > 1.795) \wedge (\text{x_pos} < 1.805) \wedge (\text{y_pos} > 0.195) \wedge (\text{y_pos} < 0.205))). \quad (28)$$

However, the natural language specifies a radius constraint around the target point, which is more directly represented as:

$$\mathbf{G}_{[20,80]}(\text{imply}(\text{z_pos} < 0.1, \sqrt{(\text{x_pos} - 1.8)^2 + (\text{y_pos} - 0.2)^2} \leq 0.05))). \quad (29)$$

The reference bounds correspond to a substantially narrower axis-aligned tolerance, suggesting a conversion or template-instantiation error. This case illustrates that, although STL-BENCH uses template control, rule-based validation, semantic deduplication, and stratified auditing, residual label errors can remain in scalable benchmark construction.

REASONSTL failure: temporal scope error. REASONSTL can still fail on nested temporal scope. Consider:

During the 120-second antenna pointing maneuver, the pitch must stay under 1.2 degrees, and if it exceeds 0.8 degrees, it must fall back below 0.5 degrees within 15 seconds.

The reference formula is:

$$\mathbf{G}_{[0,120]}((\text{pitch} < 1.2) \wedge \text{imply}(\text{pitch} > 0.8, \mathbf{F}_{[0,15]}(\text{pitch} < 0.5))). \quad (30)$$

REASONSTL predicts:

$$\mathbf{G}_{[0,120]}(\text{pitch} < 1.2) \wedge \text{imply}(\text{pitch} > 0.8, \mathbf{F}_{[0,15]}(\text{pitch} < 0.5)). \quad (31)$$

The prediction incorrectly lifts the implication outside the global temporal scope. Consequently, the recovery constraint is no longer enforced throughout the 120-second maneuver; it is evaluated only at the top level. This example shows that preserving temporal scope remains challenging for nested STL formulas.

Automatic failure but human-verified success: signal-name ambiguity. Strict automatic matching can reject semantically plausible predictions when the model uses a different canonical signal name. For a requirement referring to “rotational speed”, the reference predicate may use

$$\text{rpm} > X, \quad (32)$$

whereas model may predict

$$\text{rotational_speed} > X. \quad (33)$$

The prediction is marked incorrect by exact tree matching because the variable names differ, but the predicted signal name is semantically aligned with the requirement. This motivates human verification, signal-alias normalization, and semantic-equivalence-aware evaluation.

G Limitations

Several limitations remain. First, STL-BENCH is not exhaustively verified by human annotators. Although its construction combines template-controlled formula generation, rule-based validation, symbolic filtering, embedding-based deduplication, and stratified human auditing, not every individual sample is manually checked. As a result, residual annotation errors, ambiguous requirement interpretations, or imperfect natural-language/STL alignments may still exist.

Second, a natural-language requirement does not always correspond to a unique STL formula. Multiple specifications may be semantically reasonable depending on modeling conventions, signal abstractions, temporal granularity, and whether auxiliary assumptions are made explicit. This work favors compact and canonical formulas to support controlled training and evaluation. However, strict automatic matching may penalize alternative but semantically valid formulations. Our manual analysis also suggests that some reference formulas may be underspecified or overly canonicalized, especially for requirements involving symmetric bounds, equivalent rewritings, or alternative signal-name conventions. Future work should consider multi-reference annotations and semantic-equivalence-aware evaluation protocols.

Third, tool-augmented reasoning introduces additional inference overhead. Compared with direct generation, explicit reasoning traces and tool calls increase decoding length and require external tool execution. Although the current tool set is deterministic and lightweight, latency may remain a practical concern for interactive use or large-scale batch processing. This motivates future work on adaptive tool invocation, more efficient reasoning policies, and selective use of computation only when it is necessary.

Fourth, the current experiments are conducted primarily with a Qwen3-4B backbone due to computational constraints. Larger open-source models, such as Qwen3-8B or Qwen3-30B-A3B, may further improve performance on complex multilingual requirements, deeply nested temporal structures, and longer tool-use chains. A systematic scaling study across model sizes, training budgets, and tool-use policies is left for future work.

Finally, although REASONSTL substantially improves NL-to-STL generation, its current accuracy remains insufficient for unsupervised large-scale deployment in safety-critical CPS settings. Generated specifications may still contain subtle semantic errors, missing assumptions, overly restrictive or overly permissive constraints, or modeling choices that do not match the intended physical system. Therefore, REASONSTL should currently be viewed as a human-in-the-loop drafting assistant rather than a replacement for expert specification design. In practical use, generated STL candidates should be reviewed by domain experts, checked against system-specific modeling assumptions, and validated through downstream formal methods before deployment.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The claims in the abstract and introduction are consistent with the paper’s actual contributions and are supported by the methods and evaluation presented in the paper.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We honestly discuss the limitations of the current work in the appendix and outline possible directions for future improvement.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper primarily proposes a training framework and a multi-stage reinforcement learning pipeline, rather than presenting theoretical results or proofs. No theorems, lemmas, or formal propositions are stated, so this question is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All information necessary to reproduce the main experimental results is provided in the paper, including model architecture, training configurations and hyperparameters. Furthermore, the datasets and code framework will be made publicly available in a subsequent release, which will further facilitate reproducibility.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The datasets and code framework will be publicly released upon publication, along with sufficient instructions to faithfully reproduce the main experimental results reported in the paper.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: All relevant experimental details, including model architecture, training configurations, hyperparameters, and evaluation protocols, are thoroughly described in the experimental sections of the paper, ensuring that the main results can be fully reproduced.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper reports error bars based on the standard deviation across multiple runs with different random seeds, ensuring the statistical reliability of the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The main experiments are evaluated deterministically with temperature 0.0, and repeated deterministic decoding produces identical outputs. We therefore report deterministic accuracies in the main tables rather than error bars. Other commercial LLM APIs results are all several times average.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This research fully conforms with the NeurIPS Code of Ethics in every respect. All experiments, datasets, and methodologies have been carefully reviewed to ensure compliance with ethical guidelines, including proper attribution of existing work, responsible use of AI models, and transparent reporting of experimental results.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses both positive and potential negative societal impacts of the proposed work. On the positive side, this work advances the field of natural language to Signal Temporal Logic (STL) translation, contributing to the broader development of formal specification and verification research. Moreover, by making the full pipeline from natural language to formal STL specifications and ultimately to executable actions more practical and accessible, this work has the potential to significantly lower the barrier for deploying formal methods in real-world applications. Potential negative impacts, such as the risk of generating incorrect formal specifications that could lead to unintended system behaviors, are also acknowledged.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper poses no such risks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing assets used in this work, including prior datasets, model backbones, API models, and software libraries, are credited through citations or documentation where applicable. We respect their licenses and terms of use, and the released materials will include license and attribution information for reused assets.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper documents the construction, validation, deduplication, split protocol, tool-use annotations, and limitations of STL-BENCH. Release documentation will include data-format descriptions, licensing information, and instructions for using the benchmark and evaluation scripts.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subject experiments, and no personal, demographic, behavioral, or sensitive information is collected. Manual auditing and verification are used only to assess the semantic consistency of generated formal specifications.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human-subject experiments or collection of personal data. Manual review is limited to checking natural-language requirements and formal STL specifications, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are used as core components in this work, including local model adaptation for NL-to-STL generation, black-box API baselines, and model-assisted construction of STL-BENCH. The paper describes these roles in the method, benchmark construction, and experimental setup sections.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.